CSED 232 Object-Oriented Programming (fall 2022)

Programming Assignment #5

-GUI: Qt-

Minesweeper

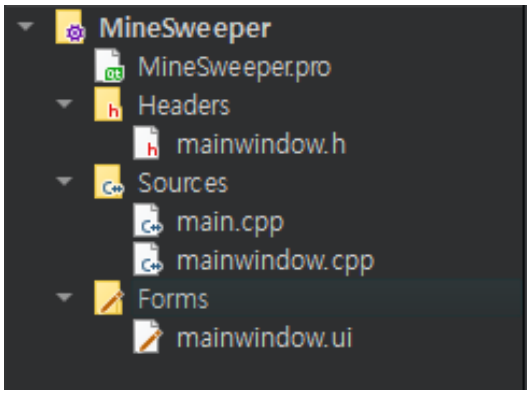
Due data: 2022년 12월 23일 금요일 23:59

담당조교: 김승욱 (wookiekim@postech.ac.kr)

다섯번째 Assignment는 학생들이 객체지향 프로그래밍을 활용하여 GUI를 가진 프로그램을 직접 만들어보도록 하는 게 목적입니다. 이를 위해 C++를 주 언어로 사용하는 GUI 프레임워크인 Qt를 사용하게 되며, 어릴적 한번쯤 플레이해보았을 (또는 누군가 플레이하는 것을 보았을) 지뢰찾기를 구현해볼 기회입니다.

● 제출 방식

- '학번'(ex: 20202121)으로 된 폴더 안에 **Qt 프로젝트 폴더 내의 모든 파일들**(*.cpp, *.h, *.pro, *.ui 등)과 제출 보고서를 포함하도록 한다



프로젝트 폴더 예시. *.pro, *.h, *.cpp, *.ui 파일을 모두 제출하도록 한다.

- 제출 보고서는 보고서 양식에 따라 필요한 내용을 중심으로 간결하게 작성하며, pdf 파일로 제출한다. 파일 이름은 '학번.pdf' 로 한다. (코드를 통째로 첨부 X. 꼭 필요한 내용만 넣을 것)

- **명예서약(Honor Code): 보고서 표지 및 작성한 코드 main마다 주석으로 다음 내용을 포함한다. "나는 이 프로그래밍 과제를 다른 사람의 부적절한 도움 없이 완수하였습니다," 없는 경우 과제를 제출하지 않은 것으로 처리한다.**

- '학번' 폴더를 '학번.zip'(20202064.zip) 파일로 묶어서 제출한다.

● 제출 방법

- PLMS(https://plms.postech.ac.kr/)의 객체지향 프로그래밍 과제메뉴에서 Assignment5 클릭
- '학번.zip' 압축파일을 첨부파일로 기간 내에 제출한다.
- **제출 기한이 지나면 하루마다 얻은 총점의 20% 감점**
  (1일 이내 지연 20% 감점, 2일 이내 지연 40% 감점, 5일 이상 지연 0점)

● 채점 방식

1) 프로그램 기능 (전체 점수 60%)

- 프로그램이 요구 사항을 모두 만족하면서 올바로 실행되는가?

2) 프로그램 설계 및 구현 (전체 점수 20%)

- 설계된 내용이 C++ 및 Qt를 이용하여 적절히 구현되었는가?
- 프로그램 설계 시 기능별로 모듈화가 잘 이루어졌는가?

3) 프로그램의 가독성 (전체 점수 10%)

- 프로그램이 읽기 쉽고 이해하기 쉽게 작성되었는가?
- 변수 명과 함수 명이 무엇을 의미하는지 알아보기 쉬운가?
- 프로그램의 소스코드를 이해하기 쉽도록 주석을 잘 붙였는가?

4) 보고서 구성 및 내용 (최대 전체 점수 10%)

- 보고서가 적절한 내용으로 이해하기 쉽게 잘 작성되었는가?
- Class 다이어그램이 포함되었고, 설계한 내용이 잘 설명되었는가?
- 실행 예제가 적절한 설명과 함께 충분히 첨부되었는가?

5) 감점 주의 사항

- 정상적으로 컴파일 되지 않을 경우, 전체 과제 0점 처리.
- 문제마다 제약사항들을 잘 확인할 것. 지키지 않을 시 감점.
- **보고서와 각 코드 main 위에 (주석으로) Honor code를 명시해야 한다. 없을 시 과제 전체점수 0점.**
- **Cheating시 F 학점은 물론 대학교 차원의 징계가 있을 것. 모든 코드는 인터넷 상의 코드와 대조해 볼 것인지 참조 및 토의 시 이해한 후 본인의 방식으로 새로 짤 것.**
- 참고한 자료들 및 사이트는 보고서에 reference 형태로 적어야 한다.
- 제출 양식(폴더 명, 파일 명, 파일 저장 위치 등)을 지키지 않을 시 과제 전체 점수 10 % 감점

● 기타

- 해당 파일에서 거론되지 않은 종류의 잘못된 입력은 주어지지 않음. 따로 거론되지 않은 부분에 대한 예외처리를 할 필요는 없으나 항상 예외처리를 고민하시는 것을 권장.
- **과제에 대한 문의 사항이 있다면 Assign5 담당 조교 이메일로 질문.**

과제 설명: Minesweeper

(목적)

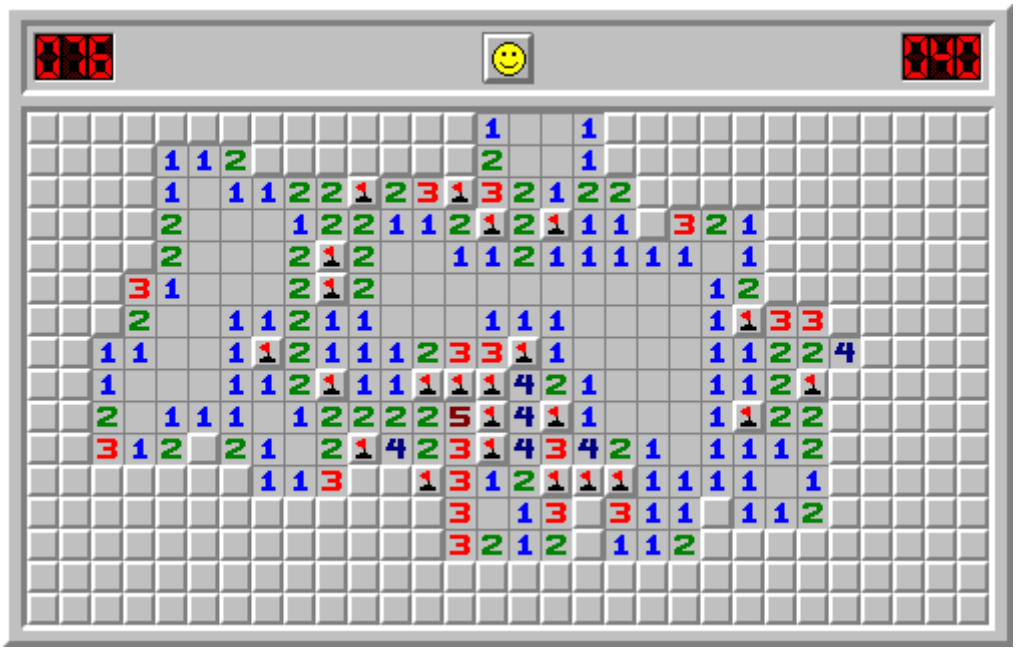
이번 과제를 통하여 C++을 사용한 GUI 프로그래밍 실습을 하도록 한다.

(주의사항)

- **문제의 출력 형식 및 GUI (버튼, 구성, 색상)는 채점을 위해 아래의 실행 예시와 최대한 비슷하게 작성해 주세요.**
- **<u>각 요소에 필요한 Qt 관련 자료는 14페이지의 구현가이드를 참고하세요.</u>**

(설명)

Qt는 C++ 중심의 객체지향 개발을 지원하는 라이브러리의 집합으로, GUI뿐만 아니라 입출력, 네트워킹, 데이터베이스 등 다양한 기능을 제공한다. Qt를 사용하면 개발자들은 C++로 개발하면서도 풍부한 GUI 애플리케이션을 쉽게 만들 수 있다. 본 과제에서는 Qt를 통해 GUI 프로그래밍을 실습해보기 위해, 간단한 GUI기반의 게임인 지뢰찾기(Minesweeper)를 구현해보도록 한다.



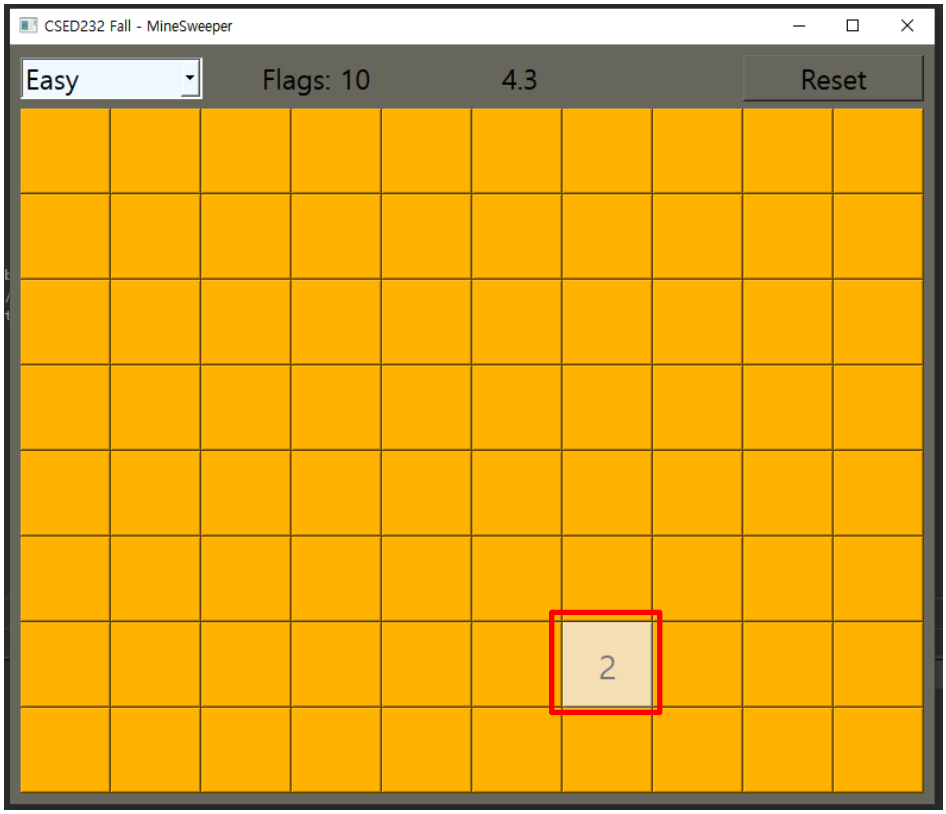
그때 그 시절 지뢰찾기

지뢰찾기의 일반적인 GUI 요소는 다음과 같다.

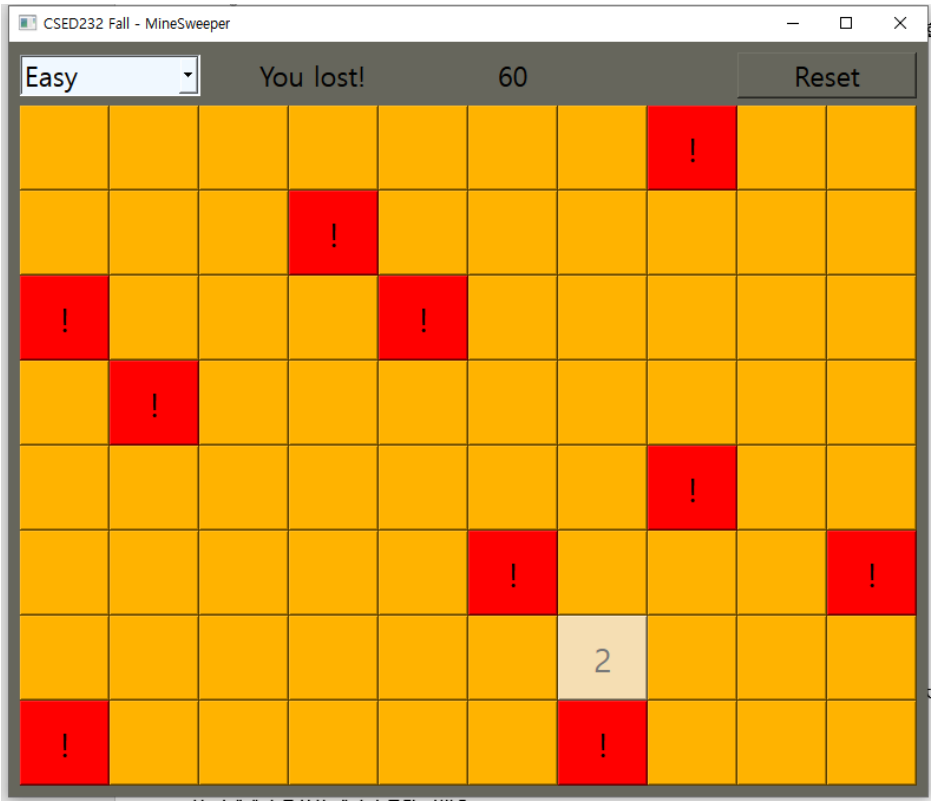
- **타이머**: 게임이 시작한 뒤 흐른 시간을 잰다.
- **남은 깃발 수**: 게임이 시작할 때 주어진 깃발의 수는 지뢰의 수와 같다. 깃발과 관련된 자세한 내용은 후술될 예정이다.
- **리셋 버튼**: 게임이 실패했거나, 다시 시작하고 싶을 때 누르는 버튼.
- **필드:** 좌클릭/우클릭을 할 수 있는 필드이다.
  - 지뢰를 좌클릭하면 게임이 실패로 끝난다.
  - 지뢰가 의심되는 자리에 우클릭으로 깃발을 세울 수 있다.

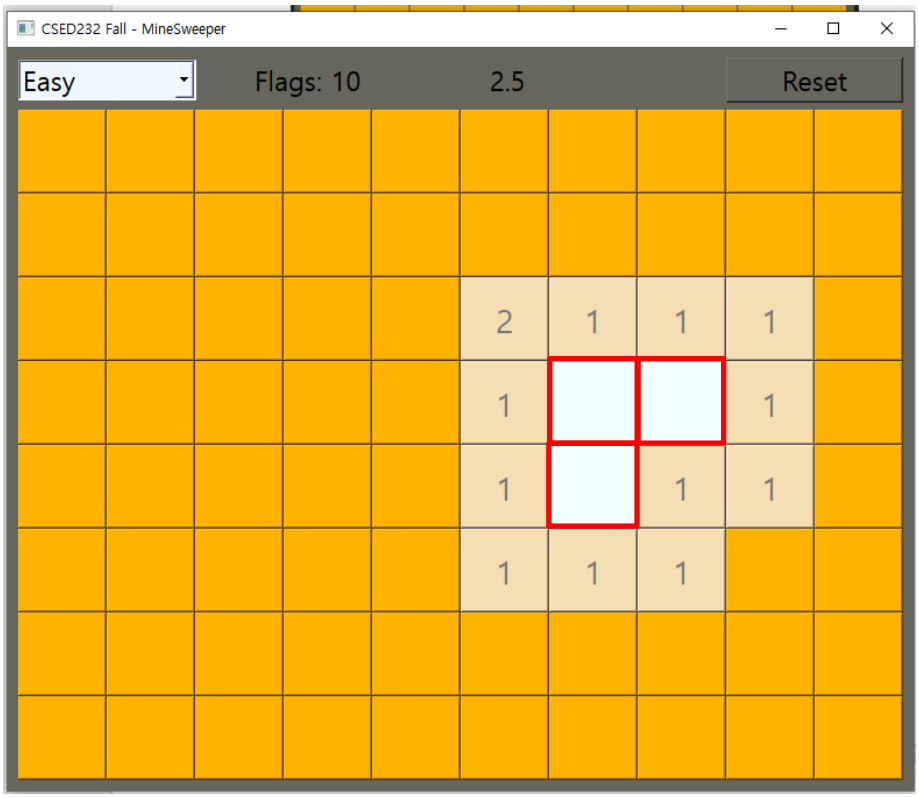
- 우클릭으로 다시 깃발을 지울 수 있다.
- 지뢰가 없지만 지뢰와 인접한 칸을 클릭하면 인접한 지뢰의 수가 뜬다.

- 지뢰도, 인접한 지뢰도 없는 칸을 클릭하면 **그 칸이 속해 있는, 테두리에 있는 칸들이 지뢰와 인접한 칸들이 모두 열린다**. 바로 하단의 예시를 참고해보자.



(지뢰가 인접한 칸을 좌클릭할 경우) 해당 칸 주변에 2개의 지뢰가 있다.





(인접한 지뢰가 없는 칸을 좌클릭할 경우) 인접한 지뢰가 있는 칸들에 둘러쌓인 칸들이 모두 열린다.

원래 지뢰찾기 게임에 있는 좌+우 동시클릭 기능은 구현하지 않는다.

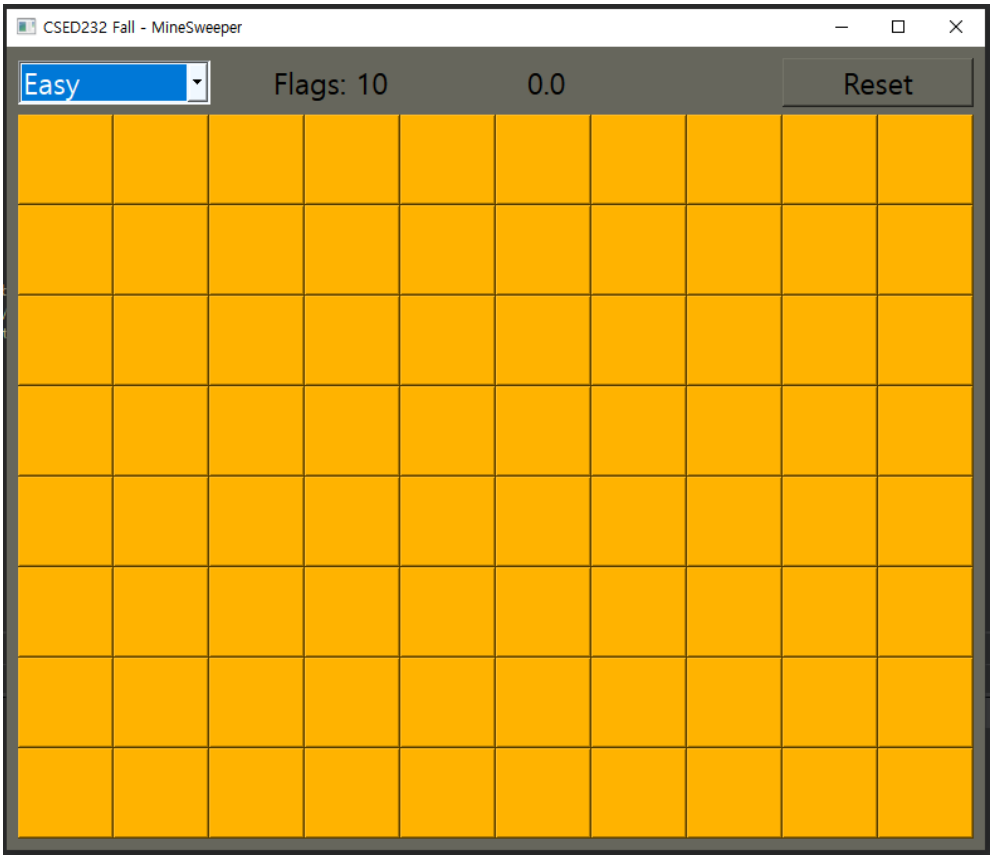
승리조건: 지뢰를 제외한 모든 칸이 열리면 승리한다. 지뢰를 좌클릭하면 지뢰가 터지면서 게임에서 패배한다.

다음 링크에서 원래의 지뢰찾기 게임을 플레이해볼 수 있다.
기본적인 룰 숙지를 위해 몇 번 플레이해볼 것을 권장한다.
https://minesweeper.online/

(구현 기능 설명 및 요구사항)

0. 기본 GUI

프로그램을 실행하면 다음과 같은 GUI창이 띄워진다.



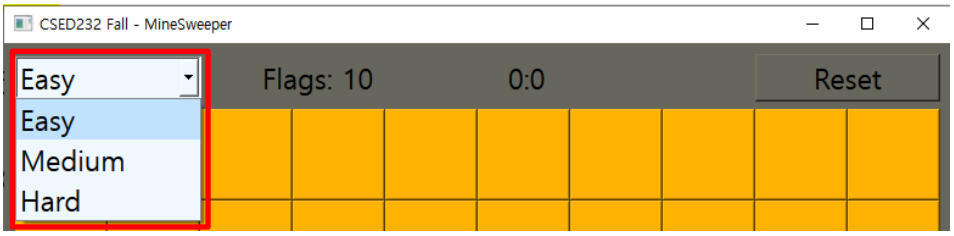
상단 메뉴는 순서대로 **난이도 선택 / 남은 깃발 수 / 타이머 / 리셋 버튼**이다. 처음 프로그램을 실행하면 기본적으로 Easy난이도 창을 띄운다.

난이도 선택

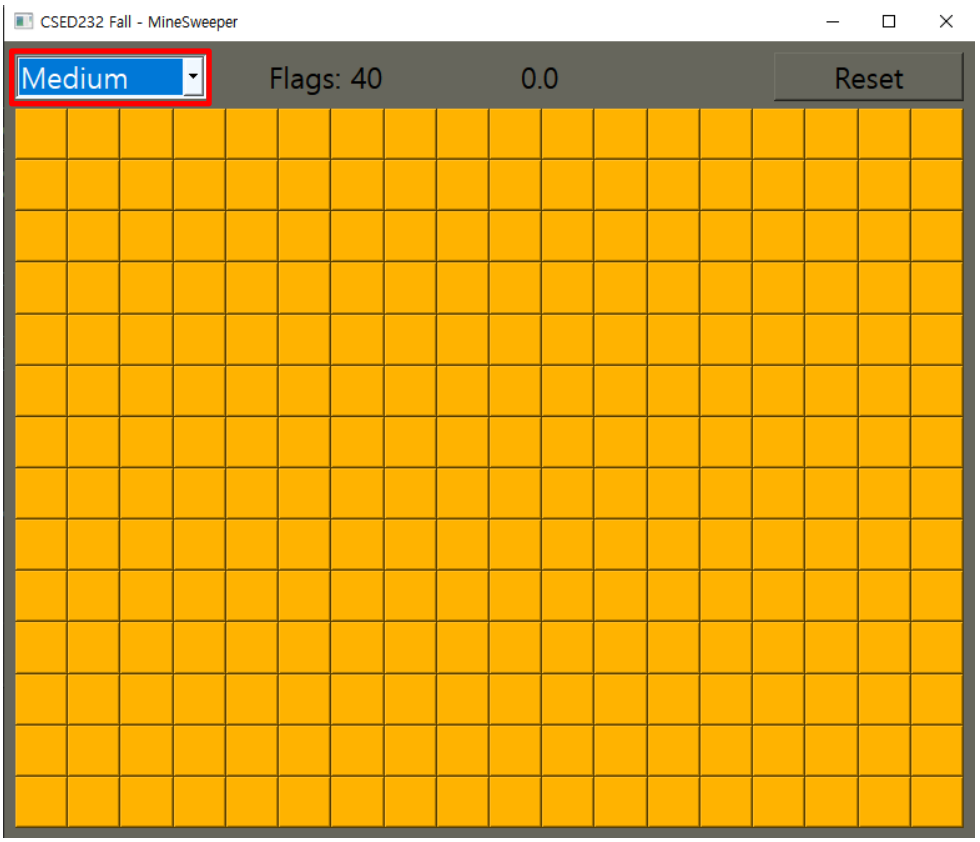
난이도는 Easy, Medium, Hard로 이루어져 있으며, 각 난이도에 따른 칸과 지뢰의 개수는 다음과 같다.

Easy: 8 x 10 칸, 지뢰 10개
Medium: 14 x 18 칸, 지뢰 40개
Hard: 20 x 24 칸, 지뢰 99개

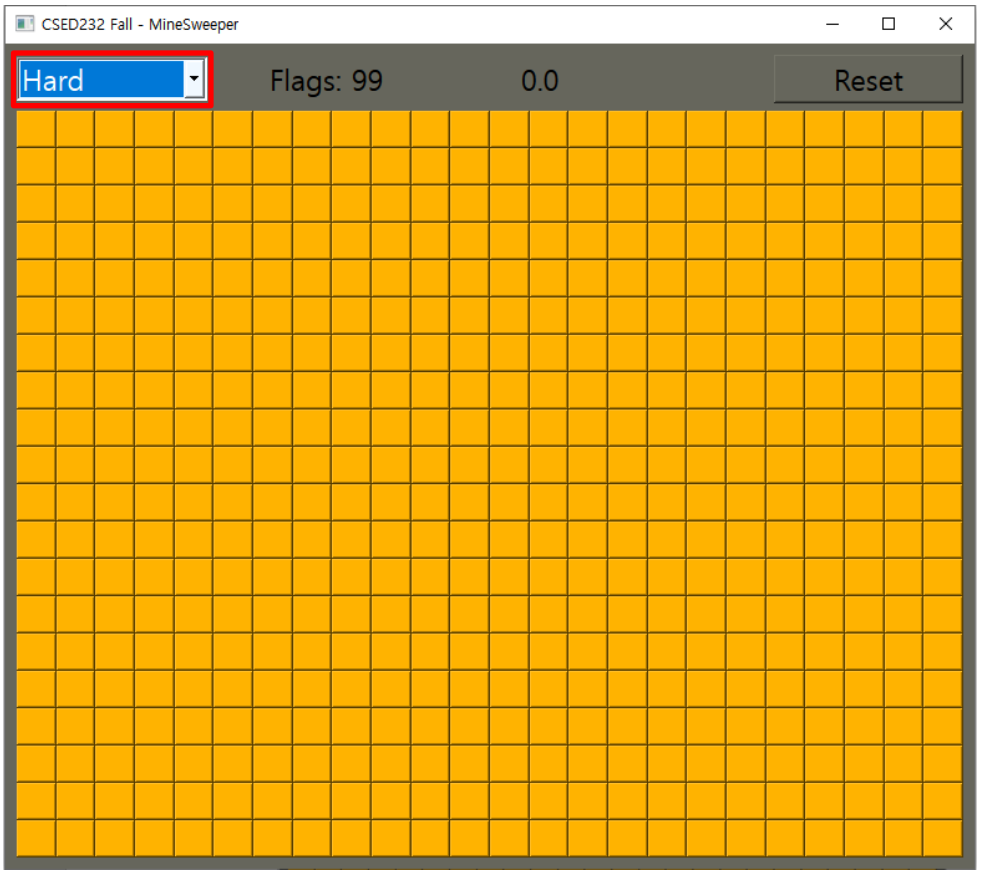
난이도에 따라 칸의 개수가 많이 차이 나기 때문에, 칸의 크기와 폰트크기를 난이도별로 상이하게 해주는 것이 좋다. 지뢰의 위치는 랜덤으로 지정되도록 한다.



난이도는 위와 같은 드롭다운 메뉴로 정할 수 있도록 한다.



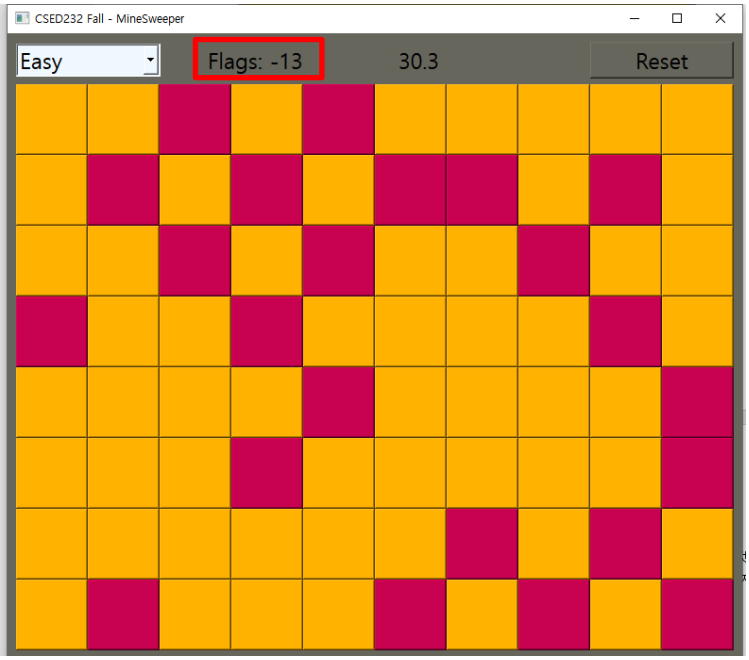
Medium 난이도 시작 화면



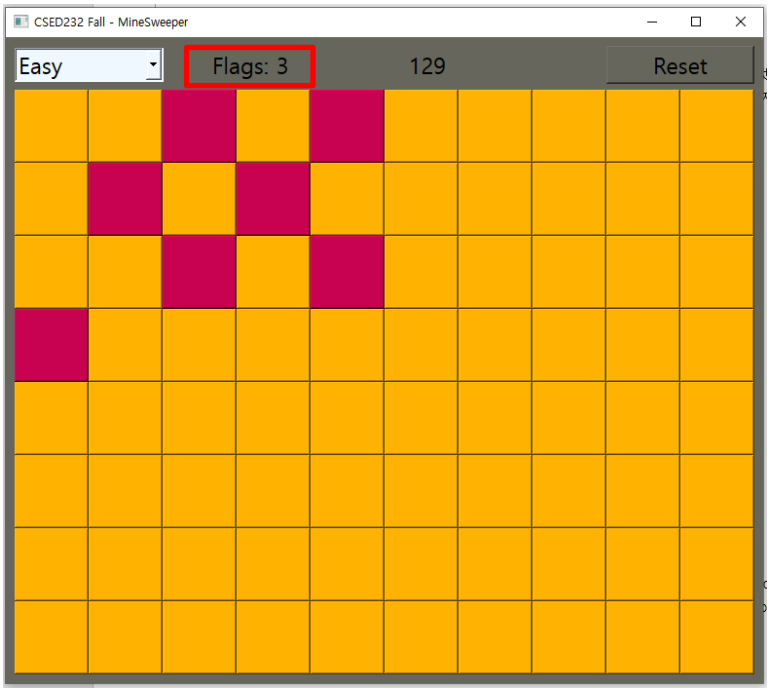
Hard 난이도 시작 화면

남은 깃발 수

게임을 시작하면 게임판에 존재하는 지뢰와 같은 수의 깃발이 주어진다. 게임을 진행하며 지뢰가 있을 것으로 생각되는 곳에 우클릭을 하여 깃발을 설치할 수 있으며, 깃발이 설치된 곳은 클릭해도 아무 일도 일어나지 않는다.



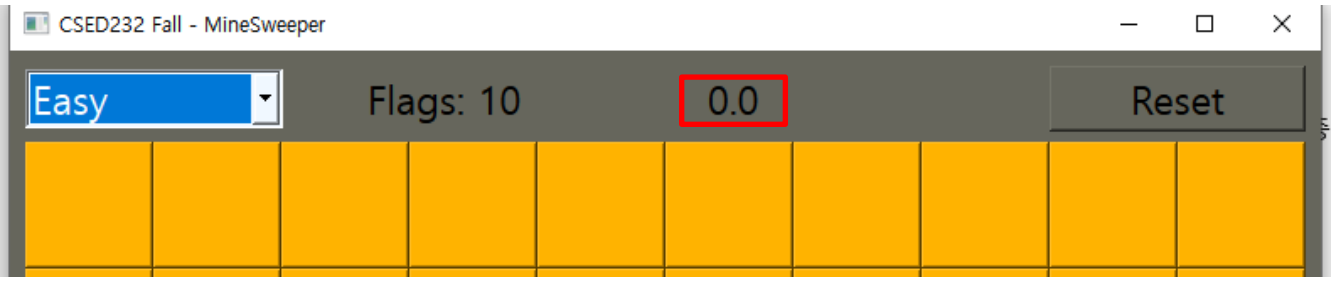
깃발은 POSTECH Red색으로 구성되어있다. 지뢰가 "의심되는" 곳에 우클릭을 하는 것이기 때문에, 주어진 깃발 개수 이상으로 깃발을 설치할 수 있다 (화면에 보면 남은 깃발 수가 음수이다). 하지만 그럴 경우 지뢰가 아닌 칸 위에 깃발이 있다는 뜻이니, 게임이 승리로 끝나지 않는다.



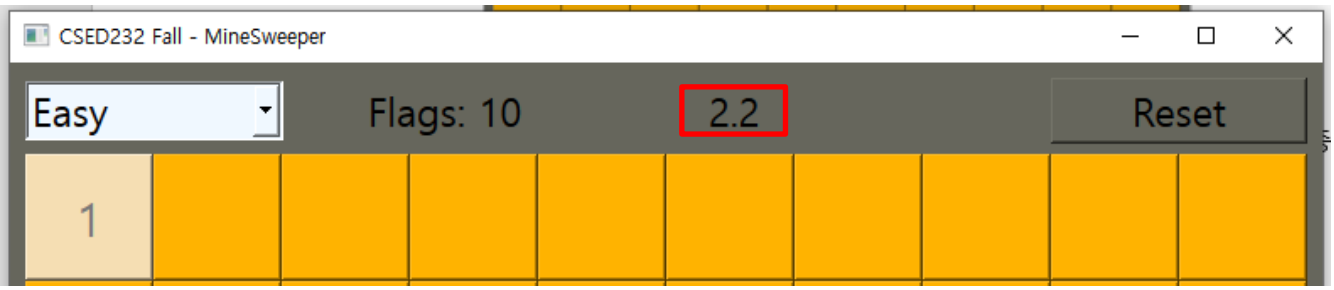
다시 우클릭을 하여 깃발을 지울 수 있으며, 설치했던 깃발을 지울 때마다 남은 깃발 수가 1씩 증가한다.

타이머

게임판 위에 클릭 (좌 또는 우)를 할 경우 타이머가 돌아가기 시작한다. 타이머는 0.1초 단위로 계산된다.

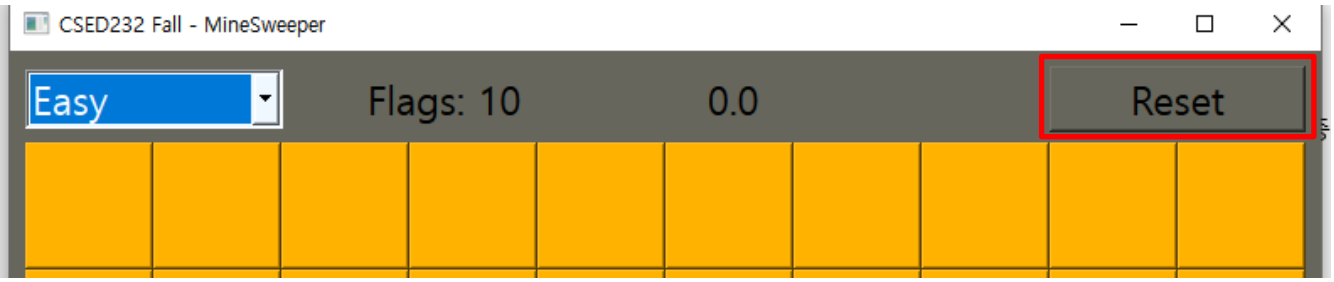


시작하기 전에 0.0초로 되어있는 타이머



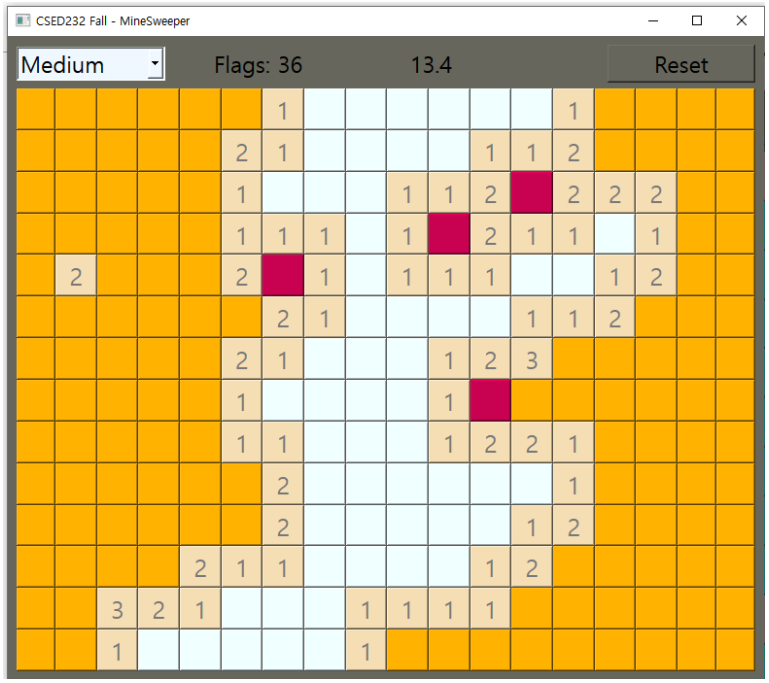
게임판 위에 클릭을 하니 증가하기 시작한다.

리셋 버튼

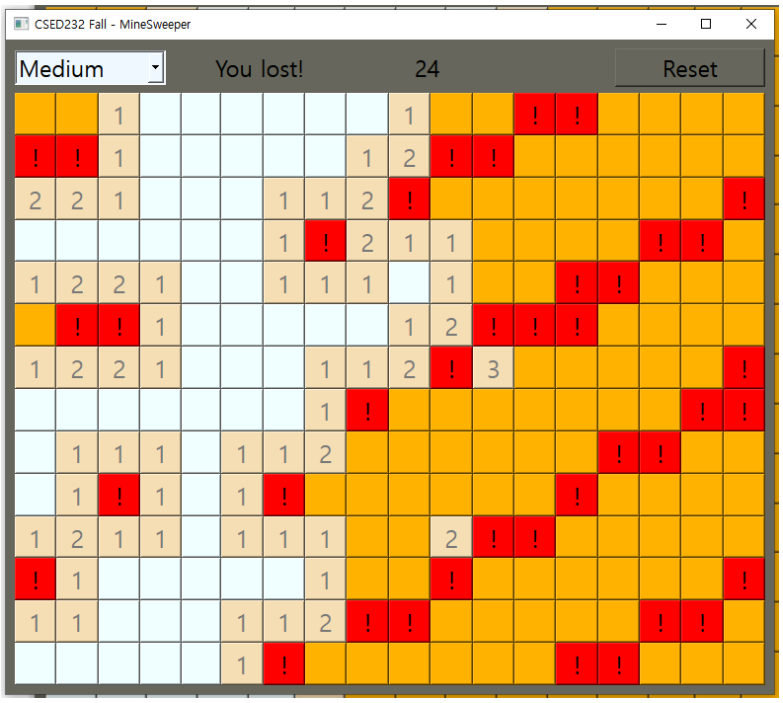


게임을 승리/패배하여 다시 시작하고 싶을 때, 또는 게임 중간에 다시 시작하고 싶을 때 리셋 버튼을 눌러서 게임판을 초기화한다. 이때, 지뢰의 위치를 재지정한다.

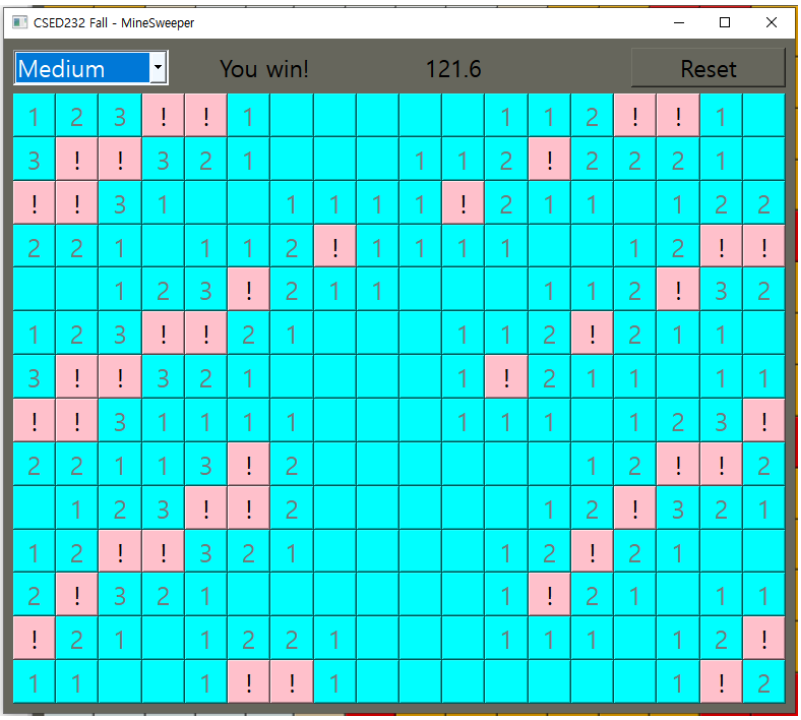
1. 실행 예시



(**게임 진행 중**) 눌리지 않은 칸, 눌렸는데 번호가 없는 칸, 눌렸는데 번호가 있는 칸, 깃발 칸에 대해 **다른 색**으로 표시됨



(**지뢰를 좌클릭하여 게임 패배시**) <남은 깃발 수> 위치에 있던 텍스트가 You lost!로 변경, 모든 지뢰의 위치가 밝혀짐.

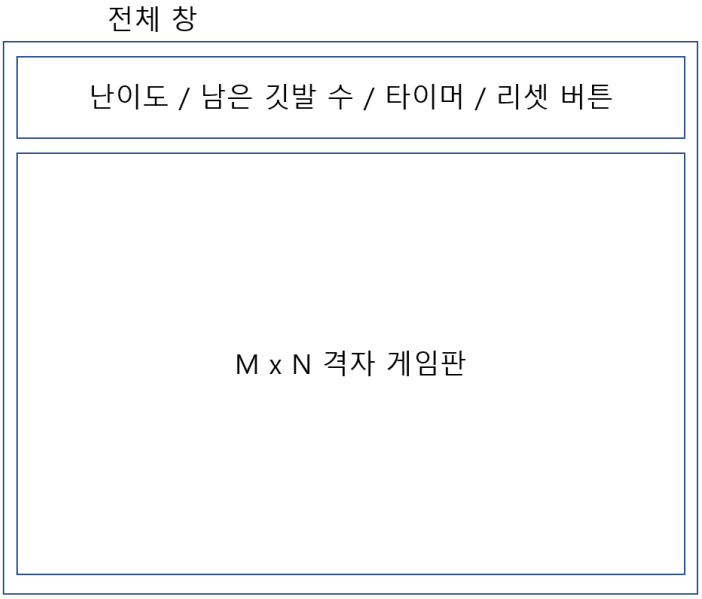


(지뢰를 제외한 모든 위치를 클릭하여 게임을 승리했을 시) <남은 깃발 수> 위치에 있던 텍스트가 You win!으로 변경, 모든 지뢰의 위치가 밝혀짐. 실패했을 때와 색이 다름.

2. 구현 가이드

GUI 구조

본 과제의 게임판 구조는 다음과 같다.



전체 창 내에 2개의 창이 세로로 쌓이는 구조이며,
첫 내부 창에서는 난이도 / 남은 깃발 수 / 타이머 / 리셋 버튼이 가로로 쌓이는 구조,
두번째 내부 창에서는 각 칸이 격자로 쌓이는 구조이다.

이를 각각 QVBoxLayout, QHBoxLayout, QGridLayout으로 구현할 수 있다.
https://doc.qt.io/qt-6/qvboxlayout.html
https://doc.qt.io/qt-6/qhboxlayout.html
https://doc.qt.io/qt-6/qgridlayout.html

난이도 선택

난이도 선택을 위한 드롭다운 메뉴는 Qt의 QComboBox를 사용하여 구현할 수 있다.
https://doc.qt.io/qt-5/qcombobox.html
connect() 함수를 통해 선택된 난이도에 따라 게임 설정을 바꾸도록 할 수 있다.
https://doc.qt.io/qt-6/signalsandslots.html

남은 깃발 개수 및 타이머

남은 깃발 개수와 게임 시작 후 지난 시간을 창에 나타내기 위해서는 QLabel을 사용할 수 있다.
https://doc.qt.io/qt-6/qlabel.html

0.1초 단위로 시간을 재는 타이머는 Qt의 QTimer를 사용하여 구현할 수 있다.
https://doc.qt.io/qt-6/qtimer.html
connect() 함수를 통해 Timer에 따라 시간을 업데이트 하는 함수를 묶어줄 수 있다.

https://doc.qt.io/qt-6/signalsandslots.html

리셋 버튼

리셋을 위한 버튼은 Qt의 QPushButton을 사용하여 구현할 수 있다.
https://doc.qt.io/qt-6/qpushbutton.html
connect() 함수를 통해 버튼이 클릭되면 게임이 초기화되도록 할 수 있다.
https://doc.qt.io/qt-6/signalsandslots.html

게임 판

게임 판의 각 칸은 마찬가지로 QPushButton을 통해 구현할 수 있다.
단, 게임 판의 버튼의 경우 좌/우클릭에 대한 처리 방식이 다르기 때문에, 우클릭을 했을 시 이를 처리할 수 있도록 QPushButton을 상속받은 새로운 MyButton 클래스를 만들어주는 게 좋다.
또한 클릭 처리를 위해 QMouseEvent를 사용할 수 있다.

https://doc.qt.io/qt-6/qpushbutton.html
https://doc.qt.io/qt-6/qmouseevent.html

```` **코드 예시: MyButton class**
```
// Initialize MyButton class
class MyButton: public QPushButton
{
    Q_OBJECT
public:
    MyButton(QWidget *parent = nullptr);
    ~MyButton();

protected:
    void mousePressEvent(QMouseEvent *e) override;

signals:
    void rightClick();
};

// override mousePressEvent
void MyButton::mousePressEvent(QMouseEvent *e){
    if(e->button() == Qt::RightButton){
        emit rightClick();
    }
    QPushButton::mousePressEvent(e);
}
```
```

각 버튼은 숫자 / 지뢰 / 빈칸일 수 있기 때문에, 각 버튼의 정보를 저장하는 구조를 따로 들고 있으면 구현이 더 편할 수 있다.
QString을 통해 해당 text (숫자, 지뢰 (!), 빈칸)을 들고 있을 수 있다.
https://doc.qt.io/qt-6/qstring.html

이후 버튼별로 connect() 함수를 통해 버튼이 클릭되면 처리하도록 할 수 있다.
https://doc.qt.io/qt-6/signalsandslots.html
``` **코드 예시: connect를 사용하여 좌/우클릭 처리**
```
connect(buttons[i][j], SIGNAL(clicked()), this, SLOT(buttonClicked()));
connect(buttons[i][j], SIGNAL(rightClick()), this, SLOT(rightButtonClicked()));

// buttonClicked(), rightButtonClicked()는 사용자 정의 함수
```

## 색상

Qt에서 색상은 setStyleSheet() 함수를 사용하여 할 수 있다.
**예1 – 색상의 이름을 사용**) w->setStyleSheet("background-color: red");
**예2 – rgb 값을 사용**) w->setStyleSheet("background-color: rgb(102,102,92)");

위 실행예제에서 사용된 색은 다음과 같다.
회색 배경: Postech Gray – rgb(102,102,92)
진한 노란색 칸 (눌리지 않음): Postech Yellow – rgb(255,179,0)
연한 노란색 칸 (눌림, 숫자 없음): azure
덜 연한 노란색 칸 (눌림, 숫자 있음): wheat
붉은 깃발: Postech Red – rgb(200,1,80)
폭탄 색 (실패시): red
폭탄 색 (성공시): pink
폭탄 외 색 (성공시): cyan
```

## 3. 기타 주의사항

- 지뢰의 위치는 반드시 랜덤으로 정해지도록 한다. 어길 시 감점이 있다.
- QT Creator에서 Run이 되지 않는다면 구현 관련 점수는 0점이 된다. 오류가 없는 코드 버전을 항상 들고 있도록 하자.
- 초기화시 (난이도를 변경하는 경우에도 초기화가 된다) 기존 게임판에 있던 칸들에 대한 동적할당 해제가 필요하다. 동적할당 해제가 되지 않았을 시 감점이 있다.
- Qt가 생소하겠지만, 온라인의 교육자료와 Qt documentation을 잘 활용하면 어렵지 않게 구현할 수 있을 것이다.